HEART DISEASE CLASSIFICATION

End-to-End MLOps Project Report

Assignment coverage

Data acquisition and EDA • Feature engineering and model development • MLflow experiment tracking • Reproducibility • CI/CD and testing • Docker API • Kubernetes deployment • Monitoring and logging

Project detail	Value
Student Name	Pavithra K
Registration Number	2025AE05504
Repository	https://github.com/pavi78/mlops-heart-disease
Dataset	UCI Heart Disease — Cleveland subset
Final Model	Tuned Logistic Regression
Submission Date	20 October 2026
Demo Video Link	MLOps_Assignment_Demo_Video.mp4

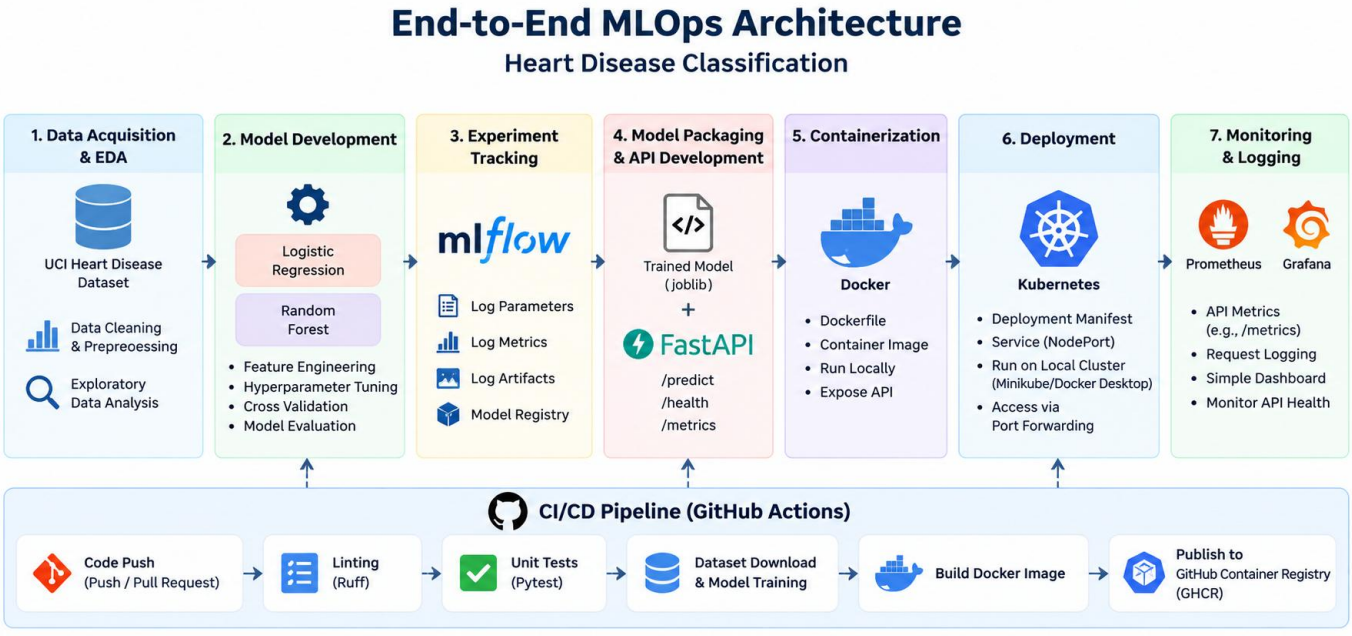


Fig 1: End-to-End MLOps Architecture

Executive Summary

This project implements an end-to-end MLOps workflow for binary heart-disease classification using the Cleveland subset of the UCI Heart Disease dataset. A download script retrieves the data, a reusable scikit-learn pipeline preprocesses features, Tuned Logistic Regression and Random Forest classifiers are compared, MLflow tracks experiments, FastAPI serves predictions, and GitHub Actions automates quality checks, training, container builds, and image publishing.

The selected model is Tuned Logistic Regression. On a held-out test set of 61 records, it achieved 88.52% accuracy, 83.87% precision, 92.86% recall, and 0.9621 ROC-AUC. The Tuned Random Forest achieved 86.89% accuracy and 0.9470 ROC-AUC.

Item	Implementation
Dataset	303 records; 14 columns
Target	Binary: num > 0 means heart-disease presence
Models	Tuned Logistic Regression and Random Forest
Validation	Stratified 80/20 split; 5-fold cross-validation
Tracking	MLflow local tracking store and registered model alias
Serving	FastAPI: /, /health, /predict, /metrics, /docs
Automation	GitHub Actions: Ruff, tests, download/train, artifacts, Docker build and GHCR publish
Deployment	Kubernetes manifests; local port-forward verified

Objectives and Scope

- Create a repeatable path from source data to model artifacts.
- Compare at least two classifiers using shared preprocessing and metrics.
- Track parameters, metrics, plots, and model artifacts with MLflow.
- Expose predictions and health information through a containerized API.
- Automate testing, training, Docker build, and image publishing.

Dataset Acquisition and Exploratory Data Analysis

The project uses the Cleveland subset distributed by the UCI Machine Learning Repository. scripts/download_data.py downloads the UCI heart-disease ZIP archive, reads processed.cleveland.data, converts '?' to missing values, assigns the expected column names, and writes data/raw/heart_disease_raw.csv.

The local dataset has 303 rows and 14 columns: age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, and num. The target is converted to binary classification using num > 0.

Data quality and preprocessing

- Missing values are represented as NaN and handled inside the preprocessing pipeline.
- Numeric features are imputed and scaled; categorical features are imputed and one-hot encoded.
- The split uses random_state=42 and test_size=0.20, producing 242 training and 61 test records.
- Stratification preserves approximate class proportions across the split.

EDA evidence

The repository contains notebooks/01_eda.ipynb. Training outputs include confusion-matrix and ROC-curve figures under artifacts/figures/.

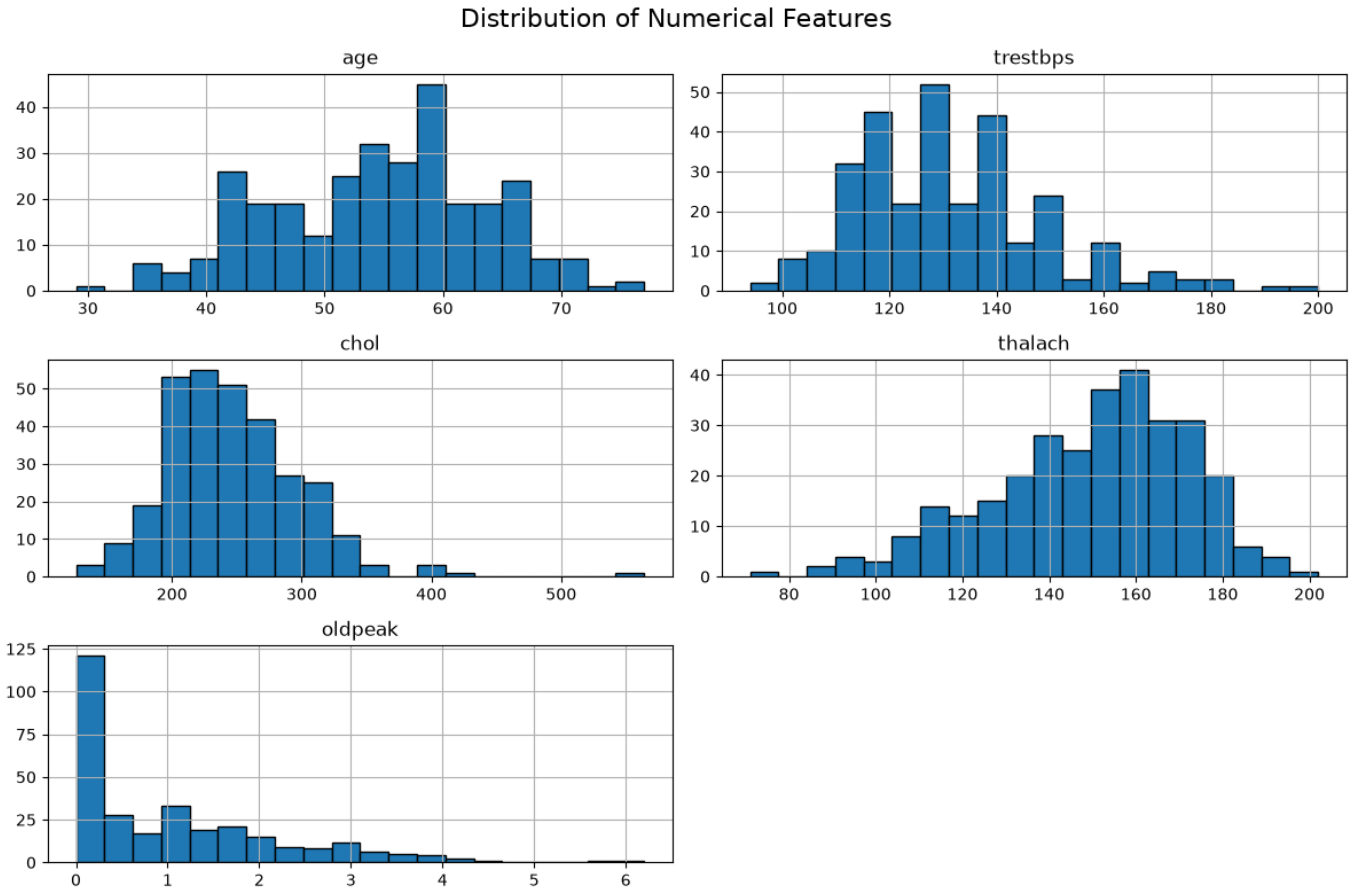


Fig 2: Distribution of Numerical Features

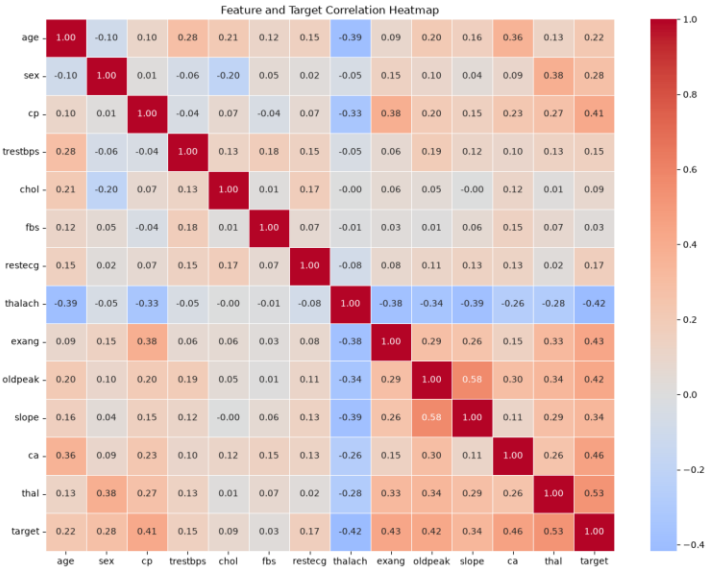


Fig 3: Feature and Target Correlation Heatmap

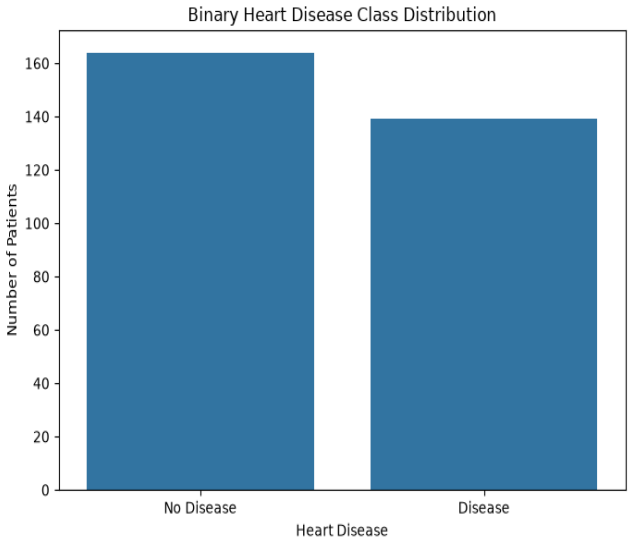


Fig 4: Binary Heart Disease Class Distribution

Feature Engineering and Model Development

The training script src/models/train.py loads the CSV, creates a reproducible train/test split, builds a preprocessing-and-classifier pipeline, tunes hyperparameters through grid search and cross-validation, compares candidate models, and saves evaluation outputs.

Tuned hyperparameters and cross-validation

Model	Best parameters	CV ROC-AUC
Logistic Regression	C=0.1; solver=liblinear	0.9117
Random Forest	max_depth=5; min_samples_leaf=1; min_samples_split=2; n_estimators=100	0.8985

Held-out test metrics

Model	Accuracy	Precision	Recall	ROC-AUC
Tuned Logistic Regression	0.8852	0.8387	0.9286	0.9621
Tuned Random Forest	0.8689	0.8333	0.8929	0.9470

Logistic Regression achieved the stronger recorded CV ROC-AUC and held-out ROC-AUC, and was selected as the final model. The serialized pipeline is artifacts/models/heart_disease_model.joblib; supporting metrics and metadata are saved under artifacts/.

Accuracy measures overall correctness; precision measures the fraction of predicted positives that are correct; recall measures the fraction of actual positives detected; ROC-AUC measures ranking discrimination across thresholds. Given the health-related domain, recall is relevant, but this educational model is not a clinical diagnostic tool.

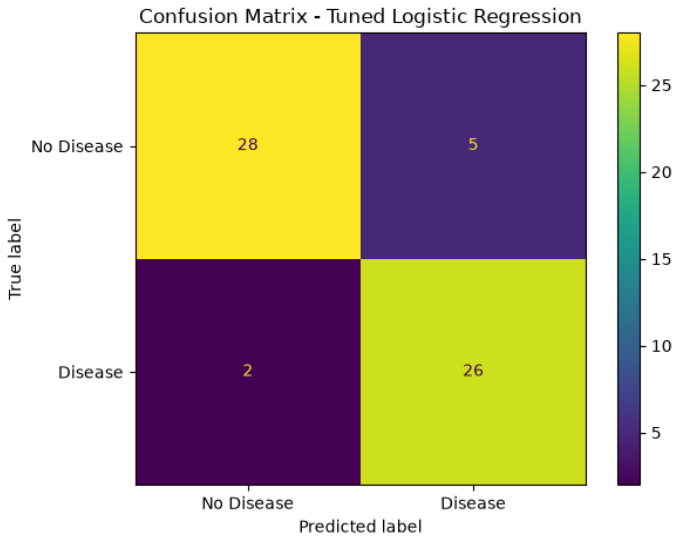


Fig 5: Confusion Matrix for TLR

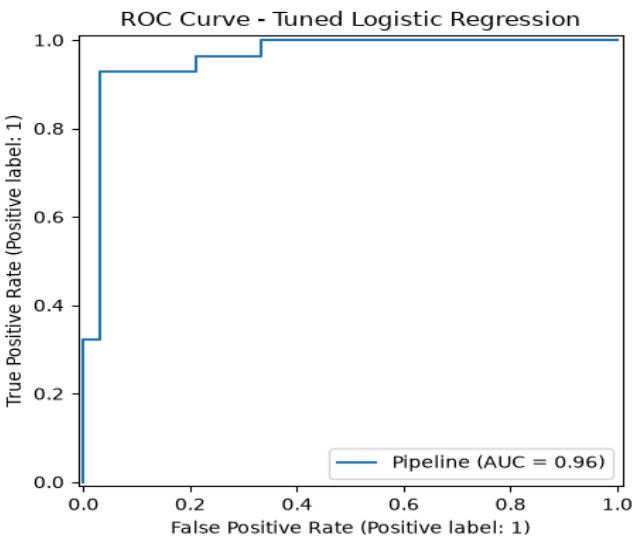


Fig 6: ROC Curve for TLR

Experiment Tracking and Artifacts

MLflow is configured with a local SQLite tracking store (mlflow.db) and the experiment name 'Heart Disease Classification'. The training workflow logs parameters, metrics, plots, and model artifacts. The project also contains an mlruns directory and a registered model named HeartDiseaseModel with the champion alias referenced by the API.

Saved artifacts

- artifacts/final_model_results.csv — comparison metrics for both tuned models.
- artifacts/training_results.csv — training and evaluation summary.
- artifacts/training_metadata.json — split settings, best parameters, CV ROC-AUC, and final metrics.
- artifacts/models/heart_disease_model.joblib — serialized scikit-learn pipeline.
- artifacts/figures/confusion_matrix_final_model.png and confusion_matrix_logistic_regression.png.
- artifacts/figures/roc_curve_final_model.png and roc_curve_logistic_regression.png.
-

Reproducibility and artifact retention

- A fixed random state (42) supports repeatable splits and stochastic steps.
- The preprocessing pipeline is bundled with the estimator so inference uses the same transformations.
- GitHub Actions uploads artifacts/ as a workflow artifact with 14-day retention.

MLflow is used for experiment organization; GitHub Actions artifacts provide downloadable output files from individual CI runs.

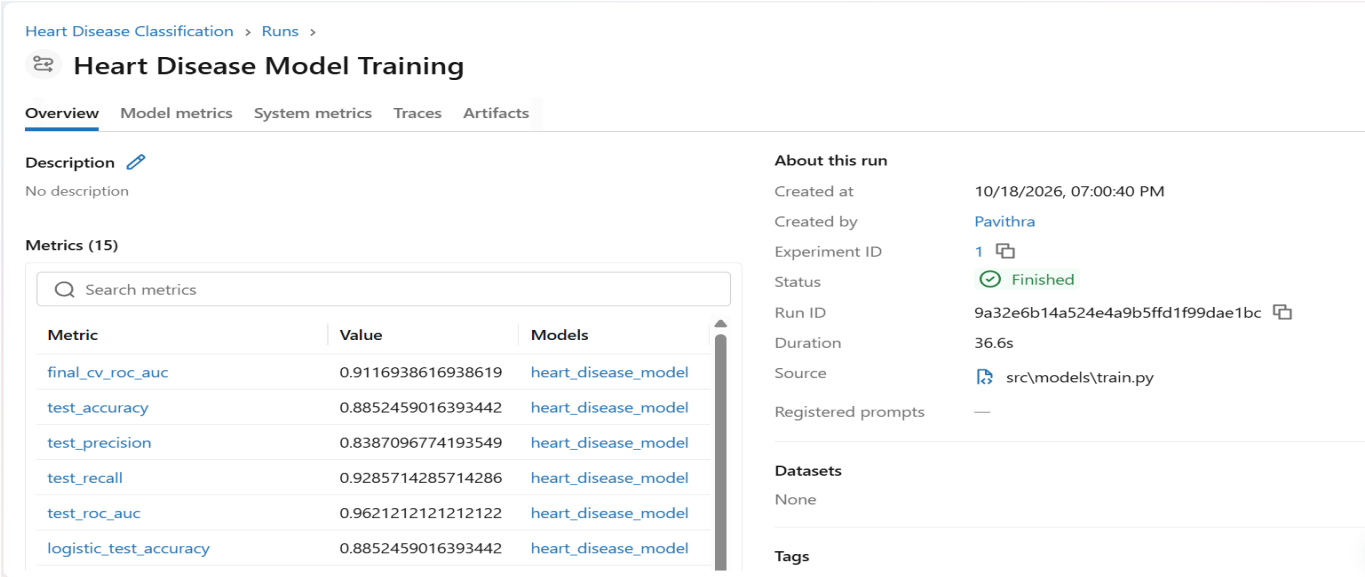


Fig 7: MLFlow Training Metrics

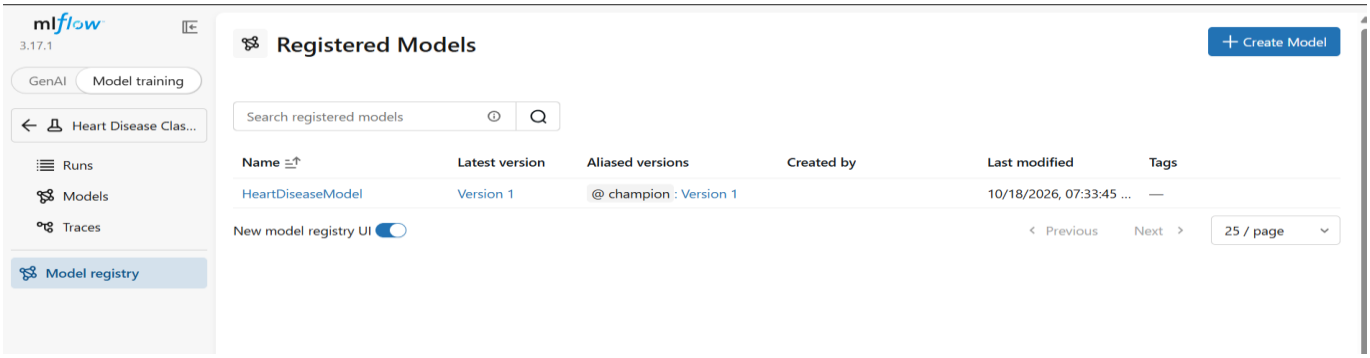


Fig 8: MLFlow Model Registry

API Packaging and Containerization

The FastAPI application in src/api/app.py exposes a root endpoint, health endpoint, prediction endpoint, and Prometheus-formatted metrics. The Dockerfile uses python:3.11-slim, installs requirements, copies src/ and model_artifacts/, sets DOCKER_MODE=true, and starts Uvicorn on port 8000.

Endpoint	Purpose
GET /	Basic API response
GET /health	Health status and model identifier
POST /predict	Accepts feature JSON and returns prediction and probability/confidence
GET /metrics	Prometheus-formatted request, latency, and process metrics
GET /docs	Interactive OpenAPI/Swagger documentation

Container verification

GitHub Actions successfully built the Docker image. Local container testing also verified prediction responses with HTTP 200, and the health endpoint reported HeartDiseaseModel@champion in the tested environment.

- docker build -t heart-disease-api:1.1 .
- docker run -d --name heart-disease-api-container -p 8000:8000 heart-disease-api:1.1
- Health check: http://127.0.0.1:8000/health
- API documentation: http://127.0.0.1:8000/docs
- Prometheus metrics: http://127.0.0.1:8000/metrics

Prediction probabilities are model outputs, not clinical risk estimates. Input feature names, units, and categories must conform to the API schema and training pipeline.

```
(.venv) PS C:\Users\Pavithra\mlops-heart-disease> docker ps
CONTAINER ID   IMAGE                                  COMMAND                  CREATED         STATUS         PORTS
                                NAMES
cbf659d67480   ghcr.io/pavi78/mlops-heart-disease:main   "uvicorn src.api.app…"   25 minutes ago   Up 25 minutes   0.0.0.
0:8000->8000/tcp, [::]:8000->8000/tcp   heart-disease-ghcr
(.venv) PS C:\Users\Pavithra\mlops-heart-disease>
```

Fig 9: Docker Container Running

Pretty-print

{"status":"healthy","model":"HeartDiseaseModel@champion"}

Fig 10: API Health Check

200

Response body

{
  "prediction": 0,
  "probability": 0.2263530805032883,
  "model": "HeartDiseaseModel@champion"
}

Download

Response headers

Fig 11: API Prediction Response

CI/CD Pipeline and Testing

The workflow .github/workflows/ci.yml runs on pushes and pull requests to main. It uses Python 3.11, installs dependencies, runs Ruff, executes pytest, downloads the dataset, trains and evaluates models, and uploads artifacts. Dependent jobs build the Docker image and publish it to GitHub Container Registry on pushes to main.

Stage	Action	Observed status
Quality and training	Ruff, pytest, dataset download, model training	Passed in workflow #11
Training outputs	Upload artifacts/; 14-day retention	Artifacts present
Docker build	Build heart-disease-api:ci	Passed
Publish	Push image to GHCR	Passed

Automated tests

The previously verified local suite contained eight passing tests: API root and health endpoints, valid and invalid prediction requests, and basic sklearn pipeline behavior for fitting, missing values, unseen categories, and metric bounds. Ruff reported 'All checks passed!'.

Pipeline design

- Docker build and publish depend on the test job, so they are blocked if the required job fails.
- Publishing is limited to pushes to main rather than pull requests.
- The successful run titled 'Add dataset download and model training to CI' (run #11) showed green test, docker-build, and publish jobs.

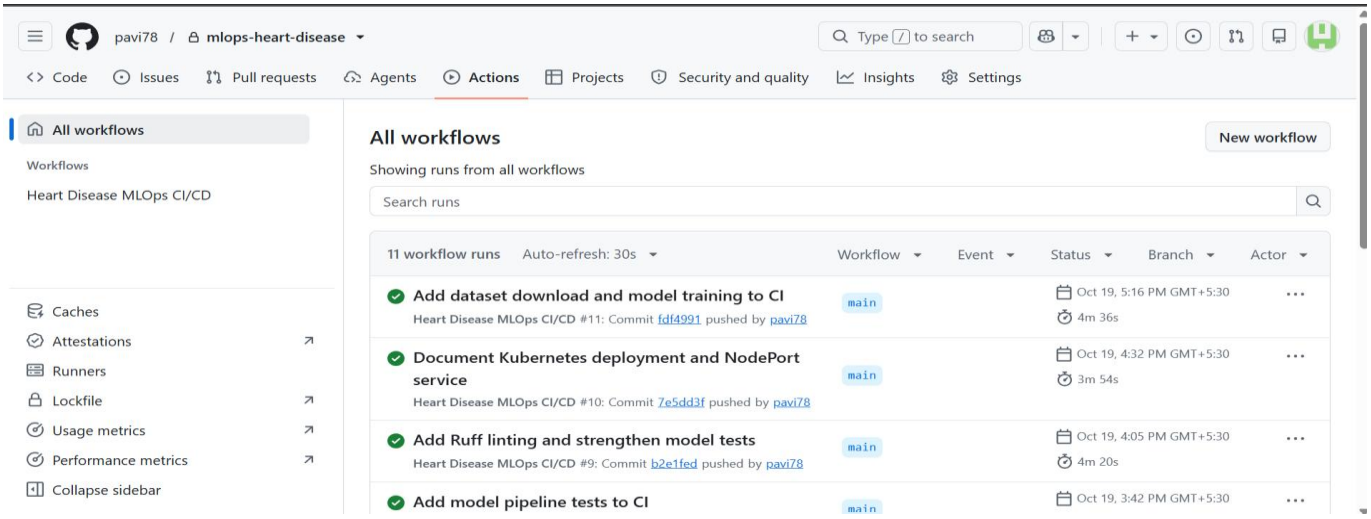


Fig 12: GitHub Commit History

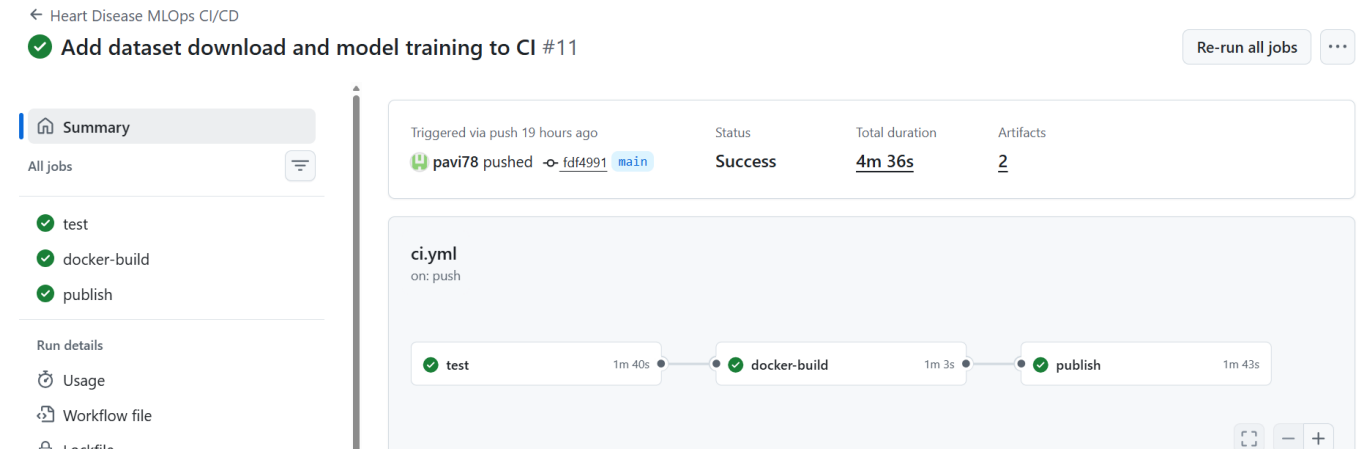


Fig 13: GitHub Action CICD

Kubernetes Deployment and Monitoring

The repository includes k8s/deployment.yaml and k8s/service.yaml. The Service is configured as NodePort on port 30080. In Docker Desktop Kubernetes, direct access through localhost:30080 returned connection refused; the verified access path was kubectl port-forward.

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods
kubectl get services
kubectl port-forward service/heart-disease-api 8002:8000
# With port-forward running:
# http://127.0.0.1:8002/health
# http://127.0.0.1:8002/docs
# http://127.0.0.1:8002/metrics
```

The pod was observed Running (1/1), and the Service endpoint was 10.244.0.6:8000. Through port-forward, /health and /docs responded successfully; a sample prediction returned HTTP 200. This is a local Kubernetes demonstration, not a public cloud deployment.

Monitoring and logging

- Prometheus FastAPI instrumentation exposes request counts, latency, request/response sizes, and Python/process metrics at /metrics.
- Application logs are written to standard output and can be inspected with kubectl logs.
- A separate Prometheus server and Grafana dashboard were not deployed.
- Current verified configuration uses NodePort plus port-forward (local-only)

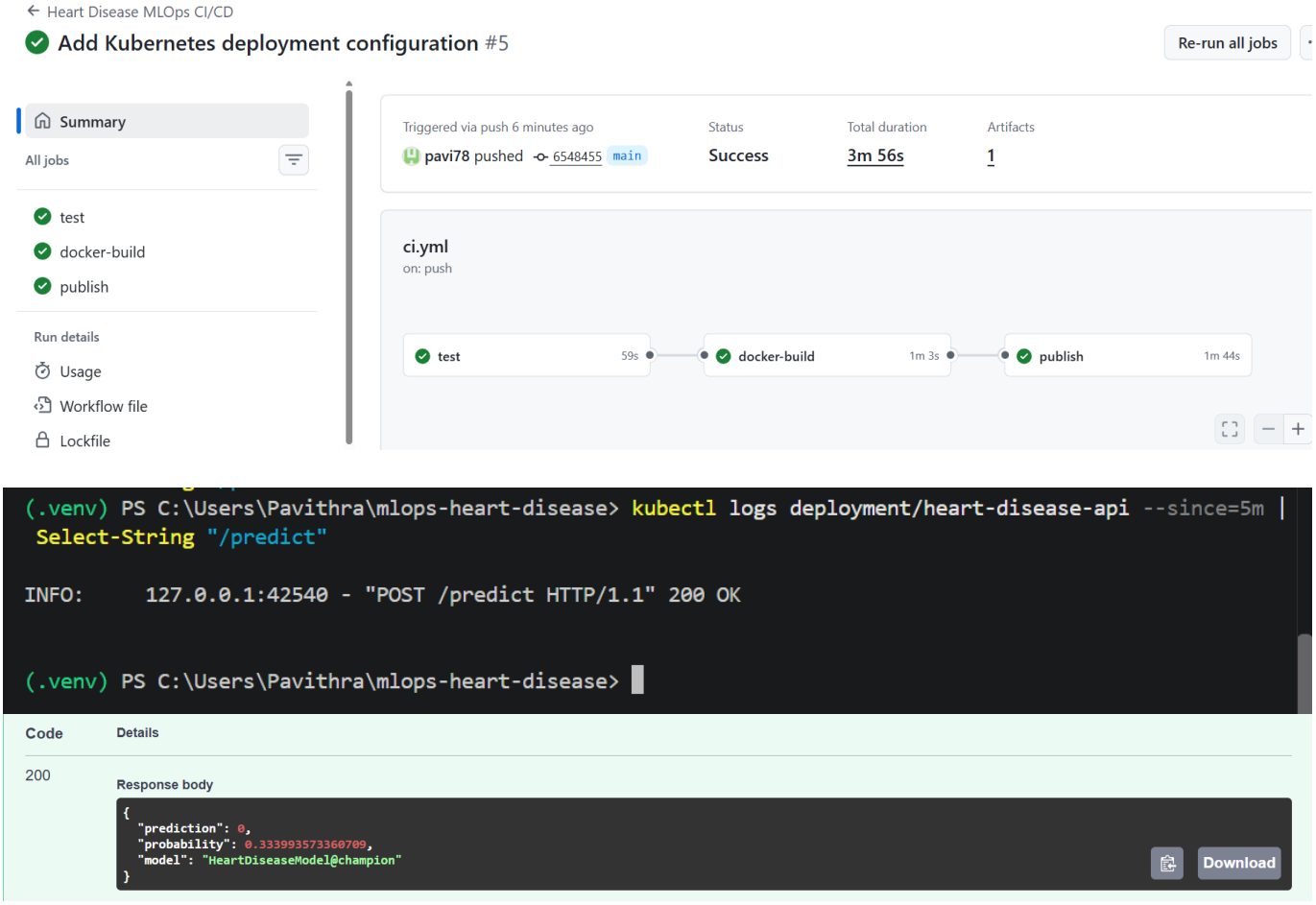


Fig 14: Kubernetes Deployment, Prediction & logs

Architecture and Reproduction

Layer	Components and flow
Data	UCI ZIP → scripts/download_data.py → data/raw/heart_disease_raw.csv
Training	src/models/train.py → preprocessing → tuned Logistic Regression and Random Forest → metrics, plots, model
Tracking	MLflow SQLite store (mlflow.db), runs, parameters, metrics, artifacts, model alias
Serving	Docker → FastAPI/Uvicorn → /predict, /health, /metrics, /docs
Automation	GitHub Actions → lint/tests → download/train → artifacts → Docker build → GHCR publish
Deployment	Kubernetes Deployment + NodePort Service → local port-forward

Reproduction commands

```
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python scripts/download_data.py
python -m ruff check src tests
python -m pytest tests/ -v
python src/models/train.py

docker build -t heart-disease-api:local .
# Verify /health, /docs, /predict, /metrics

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl port-forward service/heart-disease-api 8002:8000
```

Repository: https://github.com/pavi78/mlops-heart-disease

The raw data directory is ignored by Git but can be recreated using the download script. Main directories include src/, tests/, scripts/, notebooks/, artifacts/, model_artifacts/, k8s/, and .github/workflows/.

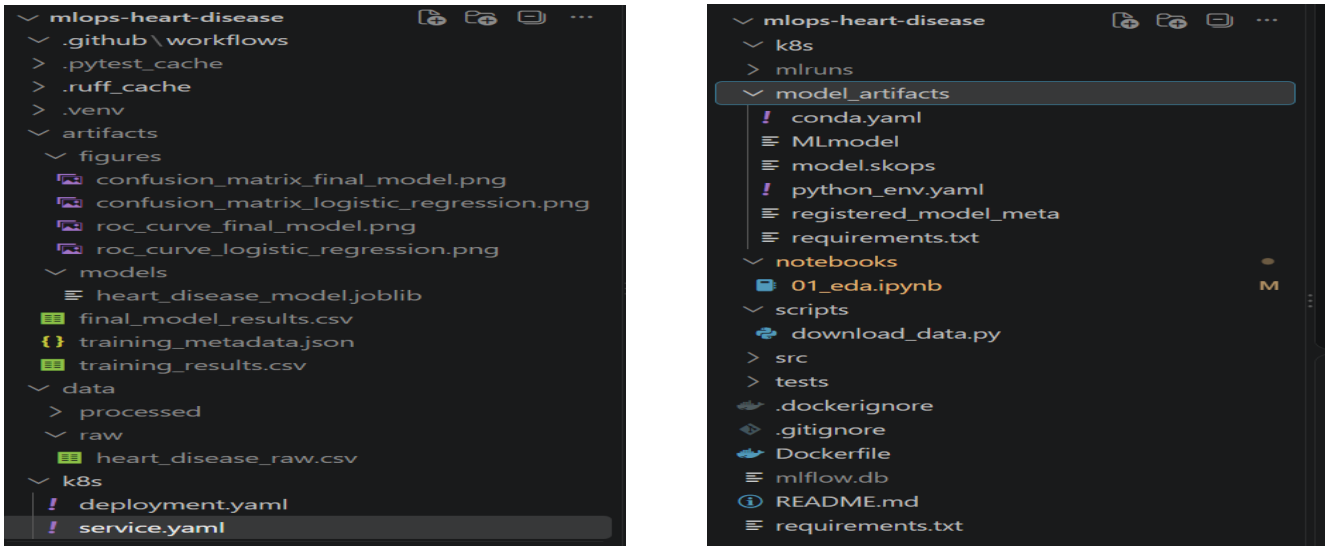


Fig 15: Project Folder Structure

Conclusion and Limitations

Conclusion

This project demonstrates an end-to-end MLOps workflow for heart disease classification, covering dataset acquisition, exploratory data analysis, data preprocessing, feature engineering, model development, experiment tracking, API development, containerization, automated testing, CI/CD, and Kubernetes deployment. Logistic Regression and Random Forest were trained and tuned using cross-validation, with Logistic Regression selected as the final model based on its evaluation results.

The selected Tuned Logistic Regression model achieved **88.52% accuracy, 92.86% recall, and a ROC-AUC of 0.9621** on the held-out test set of 61 records. MLflow was used to track experiments, parameters, metrics, and model artifacts. FastAPI exposed prediction, health-check, and monitoring endpoints, while Docker enabled containerized execution. GitHub Actions automated code quality checks, testing, model training, artifact storage, Docker image building, and publishing. The application was also verified on a local Kubernetes cluster.
Overall, the project demonstrates how MLOps practices can improve the reproducibility, automation, packaging, and operational readiness of a machine learning solution.

Limitations and Future Improvements

- **Limited dataset size:** The Cleveland heart disease dataset contains only 303 records. A larger and more diverse dataset is required to evaluate how well the model generalizes to different populations.

- **Model generalization:** The reported performance is based on a single stratified train-test split and cross-validation. Further validation on independent external datasets is needed to assess robustness and reduce the risk of overestimating performance.

- **Clinical applicability:** This model is developed for educational purposes and must not be used as a clinical diagnostic tool or to guide medical treatment without appropriate clinical validation and regulatory review.

- **Deployment scope:** Kubernetes deployment was verified locally using port-forwarding. A public cloud deployment with a verified LoadBalancer or Ingress endpoint has not been implemented.

- **Monitoring scope:** The API exposes Prometheus-formatted metrics and writes application logs to standard output. A separate Prometheus server, Grafana dashboard, and production alerting setup have not been implemented.

- **Future enhancements:** Future work could include external validation, model drift detection, automated retraining, a dedicated monitoring dashboard, security hardening, and deployment to a managed cloud Kubernetes environment.

In conclusion, the project establishes a reproducible foundation for an MLOps pipeline while identifying the additional validation, monitoring, and deployment work required before real-world production use.